

2026-05-21 Notes:

`auto_arrange_and_combine.py`

Purpose

`auto_arrange_and_combine.py` combines many already-rendered image panels into one fixed-ratio mosaic without resizing, rotating, cropping, or overlapping the input images.

The script is meant for outputs such as:

```
*_combined_VRI.png  
*_combined_R.png
```

It writes an `All...png` mosaic and a matching `.proof.txt` report. The main optimization target is:

```
Minimize the output canvas area for a requested aspect ratio X:Y.
```

Because the aspect ratio is fixed, minimizing area is equivalent to minimizing the integer scale `k` where:

```
width  = X * k  
height = Y * k
```

For example, for ratio `16:9`, the optimizer searches integer canvases:

```
16*k by 9*k
```

Current Filename

The script was renamed from the misspelled:

```
auto_arange_and_combine.py
```

to the corrected:

```
auto_arrange_and_combine.py
```

Use the corrected name in future commands.

Typical Commands

Certified/default mode:

```
/opt/miniconda3/envs/ICRAR/bin/python auto_arrange_and_combine.py -  
*combined_VRI.png 16 9
```

Fast heuristic-only mode:

```
/opt/miniconda3/envs/ICRAR/bin/python auto_arrange_and_combine.py -  
-fast *combined_VRI.png 16 9
```

Short certified attempt with fallback:

```
/opt/miniconda3/envs/ICRAR/bin/python auto_arrange_and_combine.py -  
-time-limit 60 *combined_VRI.png 16 9
```

Detailed OR-Tools search log:

```
/opt/miniconda3/envs/ICRAR/bin/python auto_arrange_and_combine.py -  
-verbose-solver *combined_VRI.png 16 9
```

Custom proof/report path:

```
/opt/miniconda3/envs/ICRAR/bin/python auto_arrange_and_combine.py \  
--proof-file VRI_16_9_arrangement_report.txt \  
*combined_VRI.png 16 9
```

If no ratio is supplied, the script uses 1:1. The legacy special ratio -1 -1 also means 1:1.

Inputs and Outputs

Inputs:

- Image files or shell/glob patterns.
- Optional trailing integer ratio `x y`, for example `16 9`.
- Existing files beginning with `ALL_`, `All_`, or `All` are skipped so prior mosaics are not accidentally re-used as inputs.

Outputs:

- A combined image, usually named from the common suffix of the inputs.
- A proof/report text file with the same stem and `.proof.txt` suffix by default.

Example output names:

```
All_combined_VRI_16_9.png  
All_combined_VRI_16_9.proof.txt
```

The script pastes each input image at native pixel size onto a black RGBA canvas. PNG output is saved losslessly with no rescaling. JPEG output is converted to RGB and saved at high quality, but JPEG is inherently lossy.

Optimization Modes

1. Certified Mode

This is the default mode.

The script first finds a feasible arrangement using deterministic heuristics. That gives an upper bound on `k`. Then it asks OR-Tools CP-SAT to prove whether any smaller `k` can work.

If OR-Tools returns:

```
OPTIMAL
```

then the output is mathematically certified as the best arrangement under the script's assumptions.

Certified means:

- all image positions are integer pixel coordinates;
- all rectangles are inside the canvas;
- no rectangles overlap;
- no smaller integer k exists for the requested ratio.

2. Fallback Mode

If OR-Tools returns a feasible arrangement but cannot prove optimality within the time limit, the script now writes the best feasible fallback layout instead of failing.

The report marks this clearly:

```
mode: OR-Tools CP-SAT fallback, not certified
status: FEASIBLE
```

This output is valid and non-overlapping, but it is not mathematically proven to be the globally best possible layout.

3. Fast Mode

`--fast` skips OR-Tools completely and writes the best layout found by the deterministic heuristic search.

The report marks this clearly:

```
mode: heuristic-only (--fast), not certified
status: HEURISTIC_ONLY
```

Use this when you need a quick mosaic and do not need proof.

Progress and Runtime Reporting

The script now prints elapsed-time progress messages like:

```
[ 0.0s] Expanding input files for ratio 16:9
[ 0.0s] Found 26 input image(s)
[ 0.2s] Finished loading images
[ 0.2s] Mathematical lower bound: k >= 310 (4960x2790)
[ 1.7s] Heuristic feasible layout: k=323, canvas=5168x2907,
method=area_chunk_shuffle_21+contact
[ 1.7s] Starting OR-Tools CP-SAT optimization with 60s time
limit
```

During a long OR-Tools solve, it prints a heartbeat every 30 seconds:

```
Still optimizing; current guaranteed fallback is k=323 (5168x2907)
```

This means the process is still alive, and if proof does not finish, the script still has a valid fallback layout to save.

The final line includes total runtime:

```
runtime      3.7s
```

Mathematical Lower Bound

For images with sizes (w_i, h_i) and requested ratio $x:y$, any valid layout must satisfy:

```
k >= max(
    max_i ceil(w_i / X),
    max_i ceil(h_i / Y),
    ceil(sqrt(sum_i(w_i*h_i) / (X*Y)))
)
```

The three terms mean:

- the canvas must be wide enough for the widest image;
- the canvas must be tall enough for the tallest image;
- the canvas area must be at least the total area of all images.

This lower bound is necessary, but not always sufficient. Sometimes the total area fits mathematically, but the rectangles cannot be arranged without overlap at that exact k . OR-Tools is used to decide that exact feasibility question.

Example from the current VRI 16:9 test:

```
lower_bound_k: 310
lower_bound_canvas: 4960x2790
fallback_k: 323
fallback_canvas: 5168x2907
```

So the fallback layout is valid, but OR-Tools did not prove whether `k=310..322` are impossible within the short test time.

Heuristic Upper-Bound Search

Before exact optimization, the script runs a deterministic MaxRects-style search. This is important because OR-Tools needs a good feasible upper bound.

The heuristic tries several item orderings:

- input order;
- largest max-side first;
- largest area first;
- smallest area first;
- widest first;
- tallest first;
- largest perimeter first;
- widest aspect ratio first;
- tallest aspect ratio first;
- most square first;
- deterministic shuffled area chunks;
- deterministic full shuffles.

For each order, it tries several placement scoring rules:

- `bssf`: best short-side fit;
- `baf`: best area fit;
- `bl`: bottom-left style placement;
- `tight`: prioritize the smallest long leftover side;
- `contact`: prefer placements contacting canvas/free-rectangle boundaries.

This heuristic is not a proof. Its job is to quickly find a compact valid layout and give OR-Tools a strong starting solution.

OR-Tools CP-SAT Model

Certified mode builds a constraint programming model.

Variables:

- k : integer scale of the output canvas;
- x_i : integer x-coordinate of image i ;
- y_i : integer y-coordinate of image i .

Canvas size:

```
width  = X * k
height = Y * k
```

Containment constraints:

```
x_i + w_i <= X * k
y_i + h_i <= Y * k
```

Non-overlap constraint:

```
AddNoOverlap2D(x_intervals, y_intervals)
```

Objective:

```
minimize k
```

Because x and y are fixed, minimizing k also minimizes canvas area.

The heuristic placement is passed to OR-Tools as a solution hint.

Proof Report

Every run writes a proof/report file.

Important fields:

- `mode`: certified, fallback, or heuristic-only;
- `status`: OR-Tools status or `HEURISTIC_ONLY`;
- `input_count`: number of input images;
- `ratio`: requested output ratio;
- `lower_bound_k`: mathematical lower bound;
- `heuristic_upper_k`: first compact feasible layout found by the heuristic;
- `final_k`: output layout scale;
- `final_canvas`: output size in pixels;
- `density`: `total_image_area / canvas_area`;
- `solver_objective_bound`: OR-Tools bound, when available;
- `solver_best_objective`: best OR-Tools feasible `k`, when available;
- `Placements`: table of file names, dimensions, and final `(x, y)` positions.

Interpretation:

- `status: OPTIMAL` means the output is certified best.
- `status: FEASIBLE` means the output is valid but not proven best.
- `status: UNKNOWN` means OR-Tools did not finish enough search to certify or improve; the script falls back to the heuristic layout.
- `status: HEURISTIC_ONLY` means `--fast` was used.

Local Dependency State

The currently validated local Python is:

```
/opt/miniconda3/envs/ICRAR/bin/python
```

Relevant package versions checked on 2026-05-21:

```
ortools 9.12.4544
protobuf 5.29.3
numpy 2.2.6
pandas 2.2.3
pillow 11.3.0
```

This matters because newer OR-Tools wheels initially imported at the top level but failed when importing `ortools.sat.python.cp_model` in this macOS conda environment.

The script includes a macOS runtime guard that re-executes itself with OR-Tools' bundled dynamic libraries first in `DYLD_LIBRARY_PATH`. This is specifically to avoid a protobuf dynamic-library mismatch when loading CP-SAT.

The environment check:

```
/opt/miniconda3/envs/ICRAR/bin/python -m pip check
```

returned:

```
No broken requirements found.
```

Validation Already Run

Syntax checks:

```
python -m py_compile auto_arrange_and_combine.py
/opt/miniconda3/envs/ICRAR/bin/python -m py_compile
auto_arrange_and_combine.py
```

Small certified case where the lower bound is attainable:

```
two 10x10 images, ratio 2:1
OPTIMAL canvas: 20x10
```

Small certified case where the area lower bound is not sufficient:

```
two 10x10 images, ratio 1:1
lower bound: 15x15
OPTIMAL canvas: 20x20
```

Temporary VRI `16:9` fallback test:

```
input_count: 26
lower_bound_k: 310
fallback status: FEASIBLE
final_k: 323
final_canvas: 5168x2907
density: 0.918547
```

This improved over the earlier heuristic result around:

```
5360x3015
```

but the short test did not prove global optimality.

Practical Recommendation

For final publication-quality mosaic generation, start with:

```
/opt/miniconda3/envs/ICRAR/bin/python auto_arrange_and_combine.py -
-time-limit 600 *combined_VRI.png 16 9
```

If it returns `OPTIMAL`, use the output as certified best.

If it returns `FEASIBLE`, the output is still valid and compact, but the proof file should be read as a fallback report rather than a mathematical proof.

If you only need a quick visual check, use:

```
/opt/miniconda3/envs/ICRAR/bin/python auto_arrange_and_combine.py -
-fast *combined_VRI.png 16 9
```

Possible Future Improvements

OpenEvolve or similar evolutionary search could be useful for improving the heuristic upper-bound stage. It should not replace OR-Tools because it cannot provide a global optimality proof by itself.

Good candidate targets for an OpenEvolve experiment:

- better `_heuristic_orders`;

- better `_placement_score`;
- post-placement local search by swapping item order;
- skyline/shelf/MaxRects hybrid strategies;
- benchmark-driven scoring across VRI, R, and synthetic edge cases.

Recommended policy:

- keep `auto_arrange_and_combine.py` deterministic and proof-aware;
- run OpenEvolve experiments in a separate experiment folder;
- only import an evolved heuristic if it improves a fixed benchmark suite and still passes strict no-overlap validation.

Important Limitations

The script optimizes rectangular image bounding boxes, not perceptual galaxy shapes. Empty black space inside each input image still counts as occupied area because the script treats every input as an indivisible rectangle.

The script does not rotate images. Rotation might improve density for some inputs, but it would change the scientific/visual orientation of panels and is not currently allowed.

Exact rectangle packing is computationally hard. For larger image sets, OR-Tools may find a very good feasible layout quickly but need much longer to prove that no smaller layout exists.